By TOUCH DEN · November 11, 2026

Virtual Machines and Containers

Video 1 · Virtualization and VMs

How Computers Work · Section 10

In this video, you will learn:

- What virtualization is
- Virtual machines and hypervisors
- Why security work relies on them

What virtualization is

One physical computer pretends to be several separate computers, each running its own OS, all isolated.

Like virtual memory (lesson 6), one level up: it gives each **OS** a private view of a whole **computer** (CPU, memory, disk), faked convincingly.

Why it matters in security:

- **Isolation:** each virtual computer is sealed off from the others and the real host.
- **Safe experiments:** run dangerous software, then throw it away.
- **Efficiency:** one server hosts many (the cloud).

Virtual machines and hypervisors

A **virtual machine (VM)** is a full, simulated computer running its **own complete OS** on top of your real one.

- Its own kernel, its own virtual disk/network/memory.
- **Strongly isolated** from the host and other VMs.

A **hypervisor** creates VMs and shares the real hardware (a manager of OSes, like the OS is a manager of programs). Tools: **VirtualBox, VMware, UTM**.

Cost: heavy, each runs a full OS, so lots of memory/disk, slow to start.

Why security relies on VMs

- **Your home lab is VMs** (course 08): an attacker machine + vulnerable targets, on an isolated virtual network. Makes attacks **safe and legal**, everything is yours and disposable.
- **Malware is detonated in VMs** (course 29): it infects the VM, not your real computer.
- **The cloud is virtualization at scale** (course 34).

The very next hands-on things you do rest on this.

Recap: virtualization and VMs

- **Virtualization:** one computer runs several isolated systems at once.
- A **VM** is a full simulated computer with its own OS; a **hypervisor** runs them.
- Your **lab**, **malware analysis**, and the **cloud** all run on VMs.

Next video: containers, snapshots, and the limits of isolation.

Virtual Machines and Containers

Video 2 · Containers, Snapshots, and the Limits of Isolation

How Computers Work · Section 10

In this video, you will learn:

- Containers vs VMs
- Snapshots and clean states
- The limits of isolation

Containers vs VMs

A **container** isolates a program but **shares the host's kernel**, no full OS of its own.

	VM	Container
Own OS kernel?	**Yes**	**No** (shares host's)
Weight	Heavy (GBs, slow)	Light (MBs, seconds)
Isolation	Very strong	Strong, but **weaker**
Best for	Different OSes, malware	Packaging/shipping apps

House (own foundation = own OS) vs **apartment** (shares the building = host kernel). Tool: **Docker**.

Snapshots: undo for a whole machine

A **snapshot** saves a VM's entire state (disk + memory) so you can **restore it instantly**.

- **Experiment without fear:** snapshot clean → break things / run malware → restore → clean again in seconds.
- **Repeatable analysis:** restore the same clean start every time.
- **Fast reset:** seconds, not an OS reinstall.

Isolation (nothing escapes) + snapshots (reset to clean) = the ideal safe playground. Your lab will snapshot before every exercise.

The limits of isolation

Isolation is enforced by **software** (hypervisor/kernel), and software has flaws.

Breaking **out** of a VM/container to the host is a **VM escape** / **container escape**, rare, difficult, prized.

- **Containers isolate less than VMs** (shared kernel), use a **VM** for truly dangerous code.
- **No isolation is absolute:** for the worst malware, add layers (dedicated machine, isolated network).
- **Patch the host and hypervisor**, isolation is only as strong as that software.

Recap: containers and limits

- **Containers** are light and fast but share the host kernel, strong but weaker isolation than a VM.
- **Snapshots** let you reset a VM to clean instantly, the heart of a safe lab.
- **Isolation is strong, not absolute:** escapes exist; layer it and patch it.

Next lesson: back to the metal, a first look at C and memory, the base for exploitation.